



Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

Faculty of Computer Science and Engineering

CS424 L-Lab Compiler Construction

Lab 09: Ambiguous and Unambiguous Grammar Using Bison

1. Lab Objective

The objective of this lab is to understand ambiguous grammar, identify ambiguity in parsing, and convert an ambiguous grammar into an unambiguous form suitable for parsing with Bison.

2. Learning Outcomes

After completing this lab, the student will be able to:

1. Explain the meaning of ambiguous and unambiguous grammar.
2. Detect ambiguity in a grammar by examining multiple parse possibilities.
3. Rewrite an ambiguous grammar into an equivalent unambiguous grammar.
4. Implement a grammar in Bison and test its behavior on sample inputs.
5. Compare the parse result of ambiguous and unambiguous rules.

3. Prerequisites

Students should already be familiar with:

- Flex basics
- Bison structure
- Tokens, grammar rules, and semantic actions
- Arithmetic expressions and operator precedence

4. Ambiguous

A grammar is called **ambiguous** if at least one valid string can produce more than one parse tree or more than one leftmost/rightmost derivation.

5. Unambiguous

A grammar is called **unambiguous** if every valid string has exactly one unique parse tree.

Ambiguity creates serious problems in compiler design because the parser cannot decide which structure to follow.

4.1 Example of Ambiguity (Step-by-Step)

Consider the grammar:

```
E -> E + E
E -> E * E
E -> id
```

Now take the input:

id + id * id

Possible Interpretation 1:

(id + id) * id

- First perform addition
- Then multiply

Possible Interpretation 2:

id + (id * id)

- First perform multiplication
- Then addition

Since both parse trees are valid → **Grammar is Ambiguous**

4.2 Removing Ambiguity Using Precedence

To fix ambiguity, we define precedence levels:

- * has higher precedence than +
- Operations follow left associativity

Unambiguous grammar:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (E) \mid \text{id}$$

Now the same input:

$$\text{id} + \text{id} * \text{id}$$

Only one valid interpretation:

$$\text{id} + (\text{id} * \text{id})$$

Ambiguity removed

4.3 Key Concepts

- **Precedence:** Defines which operator is evaluated first
- **Associativity:** Defines evaluation direction (left/right)
- **Parse Tree:** Graphical representation of derivation
- **Derivation:** Sequence of rule applications

6. Worked Example

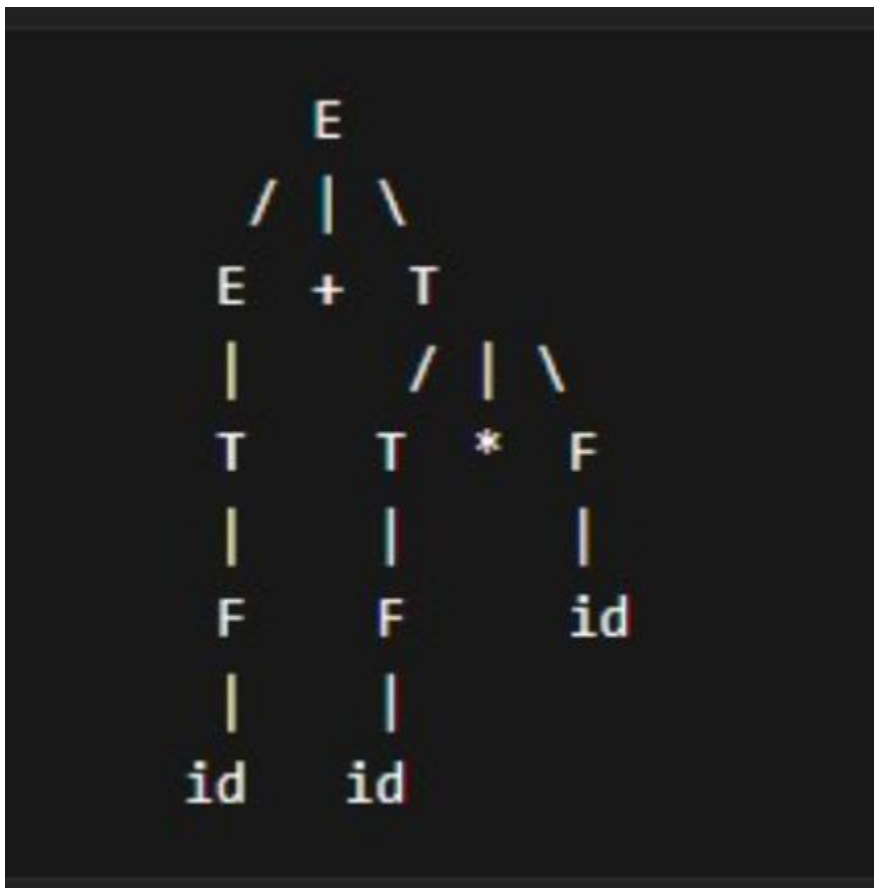
Let's manually derive the expression using unambiguous grammar:

Input:

$$\text{id} + \text{id} * \text{id}$$

Derivation:

E
 → E + T
 → T + T
 → F + T
 → id + T
 → id + T * F
 → id + F * F
 → id + id * id



This confirms:

- Multiplication handled first
- Then addition

Simple Explanation

Think of grammar like rules to solve expressions.

If rules are **unclear**, the computer gets confused → this is **Ambiguity**.

Real-Life Analogy

Expression: $3 + 4 * 5$

Two meanings:

1. $(3 + 4) * 5 = 35$

2. $3 + (4 * 5) = 23$ correct

The correct answer depends on **precedence (BODMAS rule)**.

The same happens in compilers. If grammar does not define precedence, the parser gets confused.

6.2 More Examples

Example 1

Input:

$id * id + id$

Correct Output (Unambiguous):

$(id * id) + id$

Example 2

Input:

$(id + id) * id$

Explanation:

- Parentheses force addition first
- Then multiplication

Example 3 (Invalid Input)

$id + * id$

Output:

Error: syntax error

7. Experiment 1: Identify Ambiguity

Show that the following grammar is ambiguous:

Grammar:

```
E -> E + E
E -> E * E
E -> id
```

Test it with:

id + id * id

Experiment 2: Convert to Unambiguous Grammar

Rewrite the grammar using precedence levels:

```
E -> E + T | T
T -> T * F | F
F -> (E) | id
```

Input:

id + id * id

Experiment 3: Bison Implementation

Test Inputs and Expected Outputs:

Input 1

id + id * id

Expected:

Valid Expression

Input 2

id * id + id

Expected:

Valid Expression

Input 3

(id + id) * id

Expected:

Valid Expression

Input 4 (Invalid)

id + * id

Expected:

Error

Flex File: scanner.l

```
%{
#include "parser.tab.h"
#include <stdio.h>
%}

%%

[ \t\n]+      ;
"+"           { return PLUS; }
"*"           { return MULT; }
"("           { return LPAREN; }
")"           { return RPAREN; }
[a-zA-Z_][a-zA-Z0-9_]* { return ID; }
.             { return yytext[0]; }
%%

int yywrap(void) { return 1; }
```

Bison File: parser.y

```
%{
#include <stdio.h>
#include <stdlib.h>
void yyerror(const char *s);
int yylex(void);
%}

%token ID PLUS MULT LPAREN RPAREN

%%

E : E PLUS T { printf("Reduce: E -> E + T\n"); }
  | T        { printf("Reduce: E -> T\n"); }
  ;
```

```

T : T MULT F    { printf("Reduce: T -> T * F\n"); }
  | F           { printf("Reduce: T -> F\n"); }
  ;

F : LPAREN E RPAREN { printf("Reduce: F -> (E)\n"); }
  | ID              { printf("Reduce: F -> id\n"); }
  ;

%%

void yyerror(const char *s) {
    printf("Error: %s\n", s);
}

int main() {
    printf("Enter an expression: ");
    yyparse();
    printf("Parsing completed successfully.\n");
    return 0;
}

```

Build and Run Commands

```

bison -d parser.y
flex scanner.l
gcc parser.tab.c lex.yy.c -o parser -lfl
./parser

```

Sample Input

id + id * id

Expected Output

```

Reduce: F -> id
Reduce: T -> F
Reduce: F -> id
Reduce: T -> F
Reduce: F -> id
Reduce: T -> T * F
Reduce: E -> E + T
Parsing completed successfully.

```

8. Exercises

Exercise 1: Add Subtraction (-)

Modify the grammar to include subtraction operator.

Update rule: $E \rightarrow E + T \mid E - T \mid T$

Task:

- Update Bison grammar
- Test with input: `id - id + id`

Expected: Expression should be parsed successfully

Exercise 2: Add Division (/)

Extend the grammar to support division with correct precedence.

Update rule: $T \rightarrow T * F \mid T / F \mid F$

Task:

- Update Flex and Bison files
- Test with input: `id / id * id`

Expected: Division and multiplication handled correctly

Exercise 3: Support Numbers

Modify the scanner to recognize numeric constants.

Add in Flex: `[0-9]+ { return NUM; }`

Update grammar: $F \rightarrow id \mid NUM \mid (E)$

Task:

- Test with input: `5 + 10 * 2`

Expected: Numbers should be parsed correctly